

ICEBERG AGENCY — INTERNAL ACCOUNTS MANAGEMENT SYSTEM (IAMS)

Full Architecture, Database Schema, API Spec & Core Stack Alignment Blueprint

1. EXECUTIVE REVIEW & CORE STACK ALIGNMENT AUDIT

An architectural review of the preliminary IAMS specification identified critical misalignments with the active ICEBERG codebase and deployment infrastructure. The table below outlines the disparities and how this updated blueprint reconciles them:

Architectural Domain	Preliminary Draft (Misaligned)	ICEBERG Core Stack (Aligned)	Rationale & Codebase Synchronization
Database Engine	PostgreSQL (Relational)	MongoDB (Mongoose 8.x)	Existing models (Lead , Project , CalendarSlot , Notification , MetaEvent) utilize Mongoose schemas with compound indexes and flexible subdocuments.
Object Data Modeling	Prisma ORM / Raw SQL DDL	Mongoose ODM Schemas	Native support for nested deliverables, schema-level validation, compound indexing, pre/post hooks, and serverless-safe connection pooling.
Backend Framework	Next.js App Router	Node.js + Express 4.x	The ICEBERG backend is an Express application structured in

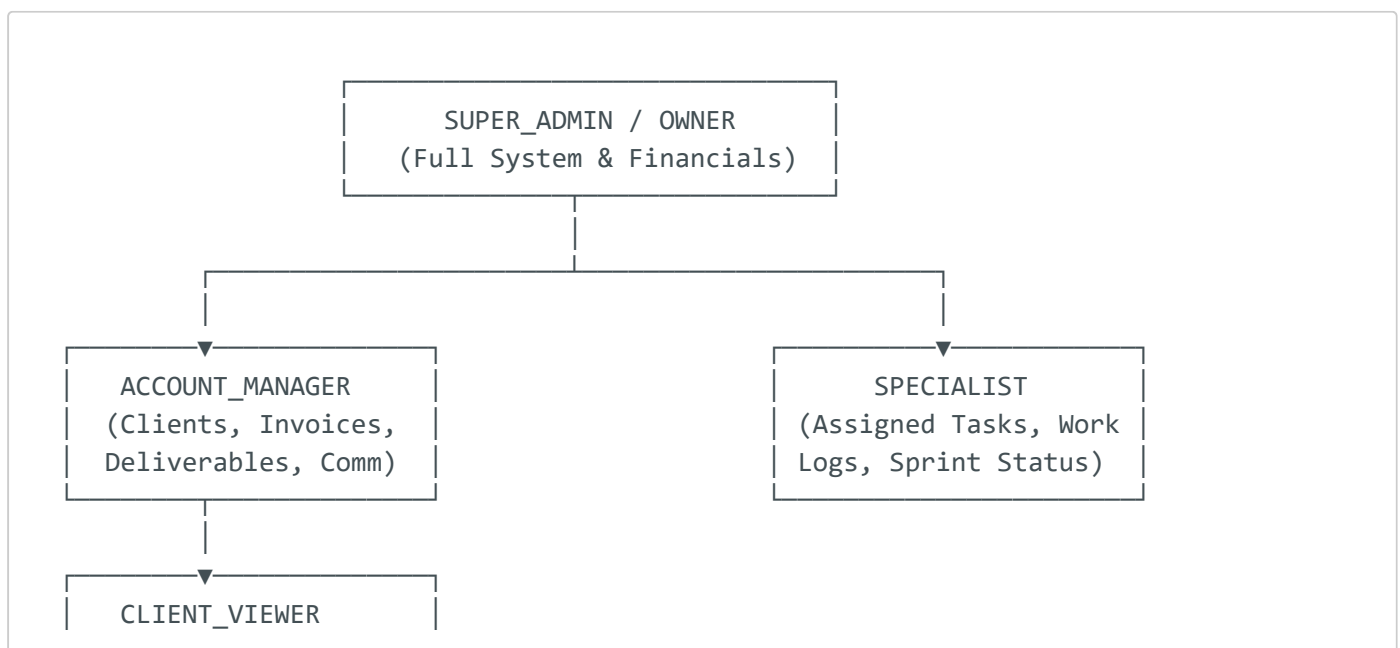
Architectural Domain	Preliminary Draft (Misaligned)	ICEBERG Core Stack (Aligned)	Rationale & Codebase Synchronization
Authentication System	Route Handlers	(api/index.js)	api/routes/* and served through Vercel serverless / Hostinger VPS (server.js).
	NextAuth.js	JWT (jsonwebtoken) + bcryptjs	Utilizes stateless JSON Web Tokens with HTTP-only cookies or Authorization: Bearer <token> headers, integrated with Express middleware.
	Standalone Next.js Application	ICEBERG Admin Suite (public/admin/)	Integrates into the current single-page dashboard architecture utilizing Tailwind CSS, Lucide Icons, Glassmorphism design tokens, and vanilla ES6 modules.
Data Resilience	Standard SQL transactions	Dual-Layer Mongo + Disk Fallback	Aligns with ICEBERG's offline/cold-start resilience pattern (leads_backup.jsonl , notifications_backup.jsonl , non-blocking buffering).
Financial Currency	Fixed USD only	Dual Currency: EGP & USD	Accommodates domestic Egyptian client operations (EGP pools/retainers) alongside international accounts (USD).
Pipeline Integration	Isolated CRM tables	Direct Lead & Calendar Ingestion	Converts qualified leads (Lead.js) and booked consultations (CalendarSlot.js) directly into active client accounts.

2. SYSTEM OVERVIEW & STACK SPECIFICATION

- **Application Type:** Internal Agency Enterprise Resource Planning (ERP), Account Management & Retainer Billing System.
 - **Target Environment:** Embedded directly within the ICEBERG Admin Console (`/admin` -> Section `#iams`) with route protection, or exposed as a secure dedicated route `/admin/iams`.
 - **Technology Stack:**
 - **Runtime:** Node.js (v18+ LTS / v20+)
 - **API Server:** Express.js (`^4.21.2`) with `helmet`, `cors`, and `express-rate-limit`.
 - **Database:** MongoDB Atlas / Local MongoDB via Mongoose (`^8.9.5`).
 - **Security & Auth:** `bcryptjs` (`^2.4.3`) for hashing, `jsonwebtoken` (`^9.0.2`) for token signing and verification.
 - **Email & Transports:** `nodemailer` (`^9.0.1`) for automated transactional notifications, invoice dispatch, and milestone alerts.
 - **Automations & Webhooks:** Internal event bus hooked into `Notification.js`, Meta CAPI sync (`api/services/metaCapi.js`), and webhook triggers for external platforms (Slack, Zapier, n8n).
 - **Frontend UI:** Vanilla ES6+ SPA controllers, Tailwind CSS (`cdn.tailwindcss.com` with configured custom palette), Lucide icons, Inter typography, and glassmorphic UI elements (`bg-slate-900/90`, border `slate-800`, accent `cyan-500` / `#06b6d4`).
-

3. ROLE-BASED ACCESS CONTROL (RBAC)

The system enforces strict multi-tenant role authorization via Express route middleware:



(Read-Only Milestones,
Deliverables & Inv)

Role	Scope & Permissions	Restricted Endpoints
SUPER_ADMIN	Complete read/write access: revenue reports, payroll, client retainers, staff assignment, system configuration, audit logs, and raw database exports.	None (Universal Access)
ACCOUNT_MANAGER	Manage assigned client portfolios, create and dispatch invoices, schedule milestones, update project budgets, and log meeting minutes.	Cannot modify staff salary records, system-wide API keys, or delete global audit logs.
SPECIALIST	View assigned projects and tasks, advance Kanban boards (TODO -> IN_PROGRESS -> IN_REVIEW -> DONE), record logged hours, and attach deliverables.	Restricted from viewing client financial retainers, invoice balances, or company-wide P&L figures.
CLIENT_VIEWER	Portal access to inspect project roadmap, approve completed deliverables, view paid/open invoices, and submit support briefs.	Strictly scoped to their own clientId; no access to internal cost models, staff rates, or Kanban task backlog.

RBAC Middleware Implementation Pattern (api/middleware/auth.js)

```
const jwt = require('jsonwebtoken');

const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'] || req.headers['x-access-token'];
  const token = authHeader && authHeader.startsWith('Bearer ') ? authHeader.split(' ')[1] : req.cookies?.iceberg_auth_token;

  if (!token) {
    return res.status(401).json({ success: false, error: 'Authentication token required.' });
  }

  try {
```

```

const decoded = jwt.verify(token, process.env.JWT_SECRET ||
'iceberg_internal_jwt_secret_2026');
req.user = decoded;
next();
} catch (err) {
return res.status(403).json({ success: false, error: 'Invalid or expired
authentication token.' });
}
};

const authorizeRoles = (...allowedRoles) => {
return (req, res, next) => {
if (!req.user || !allowedRoles.includes(req.user.role)) {
return res.status(403).json({
success: false,
error: `Access forbidden: Role '${req.user?.role || 'Guest'}' lacks
necessary permissions.`
});
}
next();
};
};

module.exports = { verifyToken, authorizeRoles };

```

4. MONGOOSE DATA SCHEMAS & DATABASE ARCHITECTURE

The database models are designed specifically for MongoDB with Mongoose, incorporating unique public string identifiers (e.g. `client_...`), full validation, lean index configurations, and support for dual currency (USD & EGP).

4.1 Users & Staff Model (`api/models/User.js`)

```

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema({
  user_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `usr_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`
  },
  email: {
    type: String,
    required: [true, 'Staff email is required'],
    unique: true,
    trim: true,

```

```

    lowercase: true,
    match: [/^\S+@\S+\.\S+$/, 'Please use a valid email address']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: 8,
    select: false // Never return password hash in regular queries
  },
  full_name: {
    type: String,
    required: [true, 'Full name is required'],
    trim: true
  },
  role: {
    type: String,
    enum: ['SUPER_ADMIN', 'ACCOUNT_MANAGER', 'SPECIALIST', 'CLIENT_VIEWER'],
    default: 'SPECIALIST'
  },
  department: {
    type: String,
    enum: ['WEB_DEV', 'SEO', 'SOCIAL_MEDIA', 'PERFORMANCE_MARKETING', 'BRANDING',
'VIDEO_PRODUCTION', 'OPERATIONS'],
    default: 'OPERATIONS'
  },
  phone: {
    type: String,
    trim: true,
    default: ''
  },
  is_active: {
    type: Boolean,
    default: true
  },
  avatar_url: {
    type: String,
    default: ''
  },
  last_login: {
    type: Date,
    default: null
  }
}, {
  timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
});

// Password hashing hook
UserSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password helper
UserSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

```

```
UserSchema.index({ email: 1 });
UserSchema.index({ role: 1 });
UserSchema.index({ is_active: 1 });

module.exports = mongoose.models.User || mongoose.model('User', UserSchema);
```

4.2 Client Accounts Model ([api/models/Client.js](#))

```
const mongoose = require('mongoose');

const ClientSchema = new mongoose.Schema({
  client_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `cli_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`
  },
  company_name: {
    type: String,
    required: [true, 'Company name is required'],
    trim: true,
    index: true
  },
  contact_person: {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, trim: true, lowercase: true },
    phone: { type: String, trim: true, default: '' },
    position: { type: String, trim: true, default: '' }
  },
  industry: {
    type: String,
    trim: true,
    default: 'General Business'
  },
  website_url: {
    type: String,
    trim: true,
    default: ''
  },
  account_manager_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  },
  status: {
    type: String,
    enum: ['PROSPECT', 'ONBOARDING', 'ACTIVE_RETAINER', 'PROJECT_BASED', 'PAUSED', 'CHURNED'],
    default: 'ONBOARDING'
  },
  financials: {
    currency: {
      type: String,
      enum: ['USD', 'EGP'],
```

```

    default: 'USD'
  },
  monthly_retainer: {
    type: Number,
    default: 0
  },
  total_lifetime_value: {
    type: Number,
    default: 0
  },
  billing_cycle: {
    type: String,
    enum: ['MONTHLY', 'QUARTERLY', 'MILESTONE_BASED'],
    default: 'MONTHLY'
  }
},
contract_period: {
  start_date: { type: Date, default: null },
  renewal_date: { type: Date, default: null },
  auto_renew: { type: Boolean, default: true }
},
originating_lead_id: {
  type: String,
  ref: 'Lead',
  default: null
},
originating_calendar_slot: {
  type: String,
  ref: 'CalendarSlot',
  default: null
},
portal_access: {
  enabled: { type: Boolean, default: false },
  user_id: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null }
},
notes: [{
  author_id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  author_name: String,
  text: String,
  created_at: { type: Date, default: Date.now }
}]
}, {
  timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
});

ClientSchema.index({ company_name: 1 });
ClientSchema.index({ status: 1 });
ClientSchema.index({ account_manager_id: 1 });
ClientSchema.index({ 'contact_person.email': 1 });
ClientSchema.index({ status: 1, created_at: -1 });

module.exports = mongoose.models.Client || mongoose.model('Client', ClientSchema);

```

4.3 Client Projects & Retainer Engagements ([api/models/AccountProject.js](#))

Note: Distinct from the public portfolio [Project.js](#) showcase model, [AccountProject](#) represents active contractual deliverables and agency operations.

```
const mongoose = require('mongoose');

const AccountProjectSchema = new mongoose.Schema({
  project_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `prj_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`,
  },
  client_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Client',
    required: true,
    index: true
  },
  title: {
    type: String,
    required: true,
    trim: true
  },
  service_category: {
    type: String,
    enum: ['WEB_DEVELOPMENT', 'SEO', 'SOCIAL_MEDIA', 'PERFORMANCE_MARKETING',
'BRANDING', 'CONTENT_PRODUCTION'],
    required: true
  },
  lead_specialist_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  },
  budget: {
    currency: { type: String, enum: ['USD', 'EGP'], default: 'USD' },
    allocated_amount: { type: Number, default: 0 },
    consumed_amount: { type: Number, default: 0 }
  },
  links: {
    repository_url: { type: String, default: '' },
    staging_url: { type: String, default: '' },
    production_url: { type: String, default: '' },
    figma_url: { type: String, default: '' },
    drive_folder: { type: String, default: '' }
  },
  timeline: {
    kickoff_date: { type: Date, default: Date.now },
    deadline: { type: Date, default: null },
    completed_at: { type: Date, default: null }
  },
  status: {
    type: String,
    enum: ['PLANNING', 'IN_DEVELOPMENT', 'IN_REVIEW', 'ACTIVE_RETAINER',
'COMPLETED', 'ON_HOLD'],
    default: 'PLANNING'
  }
});
```

```

    }
  }, {
    timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
  });

AccountProjectSchema.index({ client_id: 1 });
AccountProjectSchema.index({ lead_specialist_id: 1 });
AccountProjectSchema.index({ status: 1 });
AccountProjectSchema.index({ service_category: 1 });

module.exports = mongoose.models.AccountProject || mongoose.model('AccountProject',
AccountProjectSchema);

```

4.4 Tasks & Kanban Engine (api/models/Task.js)

```

const mongoose = require('mongoose');

const TaskSchema = new mongoose.Schema({
  task_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `tsk_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`
  },
  project_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'AccountProject',
    required: true,
    index: true
  },
  client_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Client',
    required: true,
    index: true
  },
  assigned_to_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  },
  title: {
    type: String,
    required: [true, 'Task title is required'],
    trim: true,
    maxlength: 200
  },
  description: {
    type: String,
    default: '',
    trim: true
  },
  priority: {
    type: String,

```

```

    enum: ['LOW', 'MEDIUM', 'HIGH', 'CRITICAL'],
    default: 'MEDIUM'
  },
  status: {
    type: String,
    enum: ['BACKLOG', 'TODO', 'IN_PROGRESS', 'IN_REVIEW', 'DONE'],
    default: 'TODO'
  },
  deliverable_url: {
    type: String,
    default: ''
  },
  time_tracking: {
    estimated_hours: { type: Number, default: 0 },
    actual_hours: { type: Number, default: 0 }
  },
  due_date: {
    type: Date,
    default: null
  },
  completed_at: {
    type: Date,
    default: null
  }
}, {
  timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
});

TaskSchema.index({ project_id: 1, status: 1 });
TaskSchema.index({ assigned_to_id: 1, status: 1 });
TaskSchema.index({ client_id: 1 });
TaskSchema.index({ status: 1, due_date: 1 });

module.exports = mongoose.models.Task || mongoose.model('Task', TaskSchema);

```

4.5 Financial Invoices & Retainer Ledger ([api/models/Invoice.js](#))

```

const mongoose = require('mongoose');

const InvoiceItemSchema = new mongoose.Schema({
  description: { type: String, required: true },
  quantity: { type: Number, default: 1 },
  unit_price: { type: Number, required: true },
  total: { type: Number, required: true }
}, { _id: false });

const InvoiceSchema = new mongoose.Schema({
  invoice_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `inv_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`
  },
  invoice_number: {

```

```

    type: String,
    required: true,
    unique: true,
    trim: true
  },
  client_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Client',
    required: true,
    index: true
  },
  project_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'AccountProject',
    default: null
  },
  items: [InvoiceItemSchema],
  currency: {
    type: String,
    enum: ['USD', 'EGP'],
    default: 'USD'
  },
  subtotal: {
    type: Number,
    required: true
  },
  tax_rate_percent: {
    type: Number,
    default: 0
  },
  tax_amount: {
    type: Number,
    default: 0
  },
  total_amount: {
    type: Number,
    required: true
  },
  status: {
    type: String,
    enum: ['DRAFT', 'SENT', 'PAID', 'PARTIALLY_PAID', 'OVERDUE', 'CANCELLED'],
    default: 'DRAFT'
  },
  dates: {
    issue_date: { type: Date, default: Date.now },
    due_date: { type: Date, required: true },
    paid_date: { type: Date, default: null }
  },
  payment_method: {
    type: String,
    enum: ['BANK_TRANSFER', 'INSTAPAY', 'VODAFONE_CASH', 'CREDIT_CARD', 'STRIPE',
'PAYPAL', 'CASH', 'OTHER'],
    default: 'BANK_TRANSFER'
  },
  payment_reference: {
    type: String,
    default: ''
  },
  notes: {

```

```

    type: String,
    default: ''
  }
}, {
  timestamps: { createdAt: 'created_at', updatedAt: 'updated_at' }
});

InvoiceSchema.index({ client_id: 1, status: 1 });
InvoiceSchema.index({ invoice_number: 1 });
InvoiceSchema.index({ status: 1, 'dates.due_date': 1 });

module.exports = mongoose.models.Invoice || mongoose.model('Invoice',
InvoiceSchema);

```

4.6 Activity & Audit Logs ([api/models/AuditLog.js](#))

```

const mongoose = require('mongoose');

const AuditLogSchema = new mongoose.Schema({
  log_id: {
    type: String,
    required: true,
    unique: true,
    default: () => `log_${Date.now()}_${Math.random().toString(36).substr(2, 6)}`
  },
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    default: null
  },
  user_email: {
    type: String,
    default: 'system'
  },
  action: {
    type: String,
    required: true, // e.g. "CLIENT_CREATED", "TASK_STATUS_UPDATED", "INVOICE_PAID"
    index: true
  },
  entity_type: {
    type: String,
    enum: ['USER', 'CLIENT', 'PROJECT', 'TASK', 'INVOICE', 'SYSTEM'],
    required: true
  },
  entity_id: {
    type: String,
    required: true
  },
  details: {
    type: mongoose.Schema.Types.Mixed,
    default: {}
  },
  ip_address: {
    type: String,

```

```
    default: ''
  }, {
    timestamps: { createdAt: 'created_at' }
  });

AuditLogSchema.index({ action: 1, created_at: -1 });
AuditLogSchema.index({ entity_type: 1, entity_id: 1 });
AuditLogSchema.index({ user_id: 1 });

module.exports = mongoose.models.AuditLog || mongoose.model('AuditLog',
AuditLogSchema);
```

5. REST API ENDPOINTS SPECIFICATION

All endpoints are mounted under `/api/iams` in `api/index.js` and protected by `verifyToken` and `authorizeRoles`.

5.1 Authentication & Profile (`/api/iams/auth`)

- `POST /api/iams/auth/login`: Authenticate with email/password; returns JWT and user payload.
- `POST /api/iams/auth/register`: Super Admin route to onboard staff members with roles.
- `GET /api/iams/auth/me`: Fetch authenticated user profile and team assignments.
- `POST /api/iams/auth/logout`: Invalidate sessions or clear auth cookies.

5.2 Client Accounts Management (`/api/iams/clients`)

- `GET /api/iams/clients`: Retrieve paginated client directory. Supports query filters (`?status=ACTIVE_RETAINER&am=usr_123¤cy=EGP`).
- `POST /api/iams/clients`: Create a client profile manually or convert an existing Lead.
- `GET /api/iams/clients/:id`: Retrieve comprehensive client overview (projects, active retainers, invoice balance, contact history).
- `PUT /api/iams/clients/:id`: Update client contract parameters, retainers, and assigned AM.
- `POST /api/iams/clients/:id/notes`: Add an internal account manager note.
- `POST /api/iams/clients/convert-lead/:leadId`: **Zero-friction lead pipeline bridge**. Automatically ingests data from `Lead.js` (or `CalendarSlot.js`)

to generate a new active client record.

5.3 Projects & Deliverables ([/api/iams/projects](#))

- [GET /api/iams/projects](#): List active projects filtered by category, client, or specialist.
- [POST /api/iams/projects](#): Instantiate a project under a client account with budget and deadlines.
- [GET /api/iams/projects/:id](#): Get project breakdown including roadmap milestones and budget consumption.
- [PUT /api/iams/projects/:id](#): Update staging URLs, repository links, or status.

5.4 Tasks & Kanban Engine ([/api/iams/tasks](#))

- [GET /api/iams/tasks/board/:projectId](#): Fetch Kanban column vectors (TODO, IN_PROGRESS, IN_REVIEW, DONE).
- [POST /api/iams/tasks](#): Create a new deliverable task.
- [PATCH /api/iams/tasks/:id/status](#): Lightweight drag-and-drop endpoint to transition card state and update time tracking.
- [POST /api/iams/tasks/:id/log-hours](#): Record specialist hours against project budget.

5.5 Invoicing & Retainer Ledger ([/api/iams/invoices](#))

- [GET /api/iams/invoices](#): List agency invoices with financial aggregates (Total Outstanding, Collected MTD, Overdue).
- [POST /api/iams/invoices](#): Generate a standardized invoice with auto-calculated subtotal and tax.
- [PATCH /api/iams/invoices/:id/status](#): Mark invoices as [PAID](#), [SENT](#), or [CANCELLED](#). Automatically dispatches [Notification.js](#) alert.
- [POST /api/iams/invoices/:id/send-email](#): Dispatches PDF/HTML invoice via [nodemailer](#) to the client's billing contact.

5.6 Executive Analytics & Audit Logs ([/api/iams/analytics](#))

- [GET /api/iams/analytics/summary](#): Aggregate KPIs:
 - Monthly Recurring Revenue (MRR) grouped by currency ([USD](#) vs [EGP](#)).
 - Account churn and retention rate.
 - Specialist resource capacity and logged sprint hours.
 - Pipeline conversion speed (Lead-to-Client duration).

- `GET /api/iams/analytics/audit-logs`: Chronological trail of user actions for compliance.
-

6. INTEGRATION WITH EXISTING ICEBERG SUBSYSTEMS

6.1 Lead-to-Client Ingestion Bridge

When a prospect completes the wizard form (`public/wizard-form.js`) or books an exhibition slot (`api/models/CalendarSlot.js`), the record is stored in `Lead.js`. The IAMS exposes a conversion workflow:

```
// Sample Controller logic in api/routes/iams/clients.js
router.post('/convert-lead/:leadId', verifyToken, authorizeRoles('SUPER_ADMIN',
'ACCOUNT_MANAGER'), async (req, res) => {
  const lead = await Lead.findOne({ lead_id: req.params.leadId }) || await
Lead.findById(req.params.leadId);
  if (!lead) return res.status(404).json({ success: false, error: 'Lead not found'
});

  const newClient = new Client({
    company_name: lead.company || lead.name,
    contact_person: {
      name: lead.contact_name || lead.name,
      email: lead.email,
      phone: lead.phone,
      position: lead.position || ''
    },
    industry: lead.industry || 'General',
    website_url: lead.website || '',
    originating_lead_id: lead.lead_id,
    financials: {
      currency: req.body.currency || 'USD',
      monthly_retainer: req.body.retainer || 0
    },
    status: 'ONBOARDING'
  });

  await newClient.save();

  // Mark lead status
  lead.status = '🟡 Converted to Client';
  await lead.save();

  // Trigger Notification Center bell
  await Notification.create({
    type: 'leads',
    title: 'Lead Converted to Account',
    message: `${newClient.company_name} is now an active account in IAMS.`,
```



```

    section: 'iams-clients',
    icon: 'user-check',
    color: 'text-emerald-400 bg-emerald-500/10 border-emerald-500/20'
  });

  res.json({ success: true, client: newClient });
});

```

6.2 Resilient Disk Backup Pattern

In keeping with ICEBERG's resilient data pattern (`leads_backup.jsonl`), all critical IAMS financial mutations (invoice generation, payment reconciliations) maintain an append-only disk backup log (`api/iams_audit_backup.jsonl`) to safeguard data during MongoDB maintenance or network disruptions.

6.3 Notification Center Synchronization

Events in IAMS (task assignment, invoice overdue, deliverable review) publish directly to the established `Notification.js` schema so staff receive real-time badge counts in the `/admin` header dropdown.

7. FRONTEND USER INTERFACE & ADMIN INTEGRATION

Rather than building an isolated application requiring separate build tooling, IAMS is designed to integrate seamlessly into ICEBERG's current dashboard (`public/admin/index.html` + `public/admin/admin.js`):

1. Navigation Menu Addition:

```

<!-- IAMS Navigation Section in public/admin/index.html -->
<li class="pt-3 pb-1 border-t border-slate-800">
  <span class="px-4 text-[10px] font-extrabold uppercase tracking-widest text-cyan-400">Agency Accounts (IAMS)</span>
</li>
<li>
  <a href="#" onclick="showSection('iams-overview')" class="nav-link flex items-center gap-3 px-4 py-2.5 rounded-lg text-gray-300 hover:text-white transition-all">
    <i data-lucide="briefcase" class="w-4 h-4 text-cyan-400"></i>
    Clients & Retainers
  </a>
</li>
<li>

```

```
<a href="#" onclick="showSection('iams-kanban')" class="nav-link flex
items-center gap-3 px-4 py-2.5 rounded-lg text-gray-300 hover:text-white
transition-all">
  <i data-lucide="kanban" class="w-4 h-4 text-cyan-400"></i>
  Sprint Kanban Board
</a>
</li>
<li>
  <a href="#" onclick="showSection('iams-billing')" class="nav-link flex
items-center gap-3 px-4 py-2.5 rounded-lg text-gray-300 hover:text-white
transition-all">
    <i data-lucide="receipt" class="w-4 h-4 text-cyan-400"></i>
    Invoices & Ledger
  </a>
</li>
```

2. Design Tokens & Aesthetic:

- Primary Dark Canvas: #020617 (slate-950) / #0f172a (slate-900)
- Glass Elements: rgba(30, 41, 59, 0.5) with backdrop-filter: blur(10px)
- Accents: Cyan #06b6d4 (cyan-500), Emerald #10b981 (active retainers/paid), Amber #f59e0b (in-review), Rose #f43f5e (overdue/critical).

8. IMPLEMENTATION DIRECTORY STRUCTURE

ICEBERG/

api/

index.js

routes)

middleware/

auth.js

models/

User.js

Client.js

AccountProject.js

Task.js

tracking

Invoice.js

AuditLog.js

ICEBERG AGENCY – INTERNAL ACCOUNTS MANAGEMENT SYSTEM (IAMS).md

routes/

iams/

index.js

auth.js

clients.js

projects.js

Root Express app (mounts /api/iams

JWT verification & RBAC authorization

Staff & client auth model

Client accounts & retainer terms

Operational deliverables & repos

Kanban tasks & specialist time

Invoices, tax, and payment ledger

System-wide compliance & action logger

Aggregated IAMS router entry point

Staff login & user administration

Accounts CRUD & Lead conversion

Contract project deliverables

	tasks.js	# Kanban drag/drop & time logs
	invoices.js	# Financial billing & payments
	analytics.js	# MRR, burn rate, and audit trails
public/		
admin/		
index.html		# Integrated IAMS dashboard view panels
iams.js		# Modular client-side IAMS controller

9. VERIFICATION & SECURITY CHECKLIST

- **✓ Database Engine:** Standardized exclusively on MongoDB / Mongoose 8.x, eliminating PostgreSQL/Prisma mismatch.
- **✓ Stateless Security:** Express JWT authentication with RBAC role guards on every endpoint.
- **✓ Dual Currency:** Native support for EGP & USD retainer contracts and invoice itemization.
- **✓ Data Pipeline:** Direct conversion of inbound IDEX/Marketing leads into formal client accounts.
- **✓ Resilience:** Fallback file logging ([api/iams_audit_backup.jsonl](#)) protecting financial transactions.
- **✓ Frontend Harmony:** Complete UI consistency with ICEBERG's Tailwind CSS glassmorphic aesthetic.